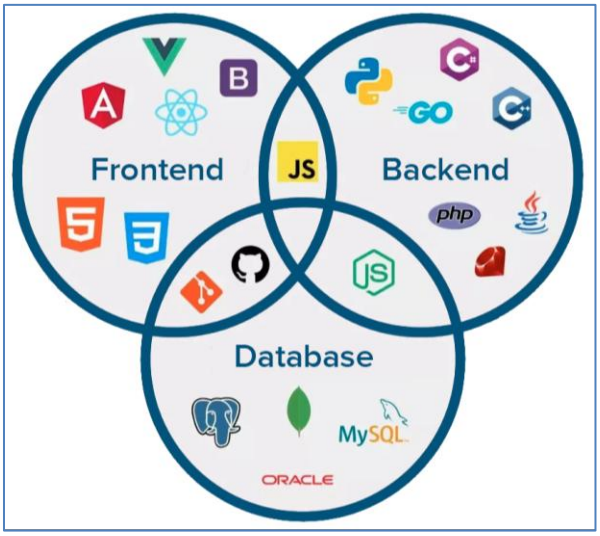
Encontro 1 – Ambiente de desenvolvimento Vue.js

Sumário

Encontro 1 – Ambiente de desenvolvimento Vue.js 1
1. Stack do Ambiente de Desenvolvimento 2
   1.1. Conceito geral 2
   1.2. Exemplo de stack moderna com Vue.js 3
   1.3. Stack local vs. Stack containerizada 3
   1.4. Importância do stack 5
2. Requisitos tecnológicos 7
   2.1. Comparação entre modelos com e sem Docker 7
   2.2. Ambiente sem Docker 7
   2.3. Ambiente com Docker 8
   2.4. Quando escolher cada abordagem 9
   2.5. Dicas práticas para ambos os ambientes 9
3. Guia para Stack “Dockerizada” 9
   3.1. O que é o WSL2 e como configurá-lo no Windows 10/11 10
   3.2. Instalação do Docker Desktop no Windows 10
   3.3. Principais Comandos Docker 11
   3.4. Docker Hub e Interação pelo Docker Desktop 11

1. Stack do Ambiente de Desenvolvimento



O termo "Stack do Ambiente de Desenvolvimento" refere-se ao **c**onjunto de tecnologias, ferramentas e configurações que compõem a base de trabalho de um desenvolvedor — ou seja, tudo o que é necessário para criar, executar, testar e manter um projeto de software de forma eficiente e padronizada. É o alicerce técnico sobre o qual um projeto de software é construído.

Quanto mais consistente, documentado e automatizado for o stack, maior a produtividade e a qualidade do trabalho da equipe.

1.1. Conceito geral

O stack (ou "pilha") é uma combinação organizada de camadas tecnológicas. Em um ambiente de desenvolvimento, ele define:

- Linguagem principal usada (ex.: JavaScript, Python, C#);
- Frameworks e bibliotecas (ex.: Vue.js, React, Django);
- Ferramentas de build e empacotamento (ex.: Vite, Webpack);
- Gerenciadores de dependências (npm, Yarn, pnpm);
- Sistemas de controle de versão (Git, GitHub);
- Ambiente de execução (Node.js, Docker, WSL2, etc.);
- Ferramentas auxiliares (ESLint, Prettier, Tailwind, Vitest, Playwright);
- Editor ou IDE configurado (VS Code, com extensões e Dev Containers, por exemplo).

Cada camada da stack trabalha em conjunto, garantindo que todo o ciclo de desenvolvimento — do código à execução — ocorra de forma previsível, independente da máquina ou do sistema operacional.

1.2. Exemplo de stack moderna com Vue.js

Um ambiente Vue.js atual costuma incluir:

Camada	Tecnologia	Função
Frontend Framework	Vue 3	Base reativa da aplicação
Build Tool	Vite	Servidor de desenvolvimento rápido e empacotador
Gerenciador de pacotes	npm / pnpm	Instala e gerencia dependências
Estilização	Tailwind CSS	CSS utilitário para prototipagem rápida
Gerenciamento de estado	Pinia	Controle global de dados
Roteamento	Vue Router	Navegação entre páginas
Testes	Vitest / Playwright	Testes unitários e end-to-end
Padronização de código	ESLint + Prettier	Corrige e formata automaticamente
Controle de versão	Git + Husky/lint-staged	Garante commits limpos e testados
Ambiente isolado	Docker + VS Code Dev Containers	Reproduzível e padronizado para todos os devs

1.3. Stack local vs. Stack containerizada

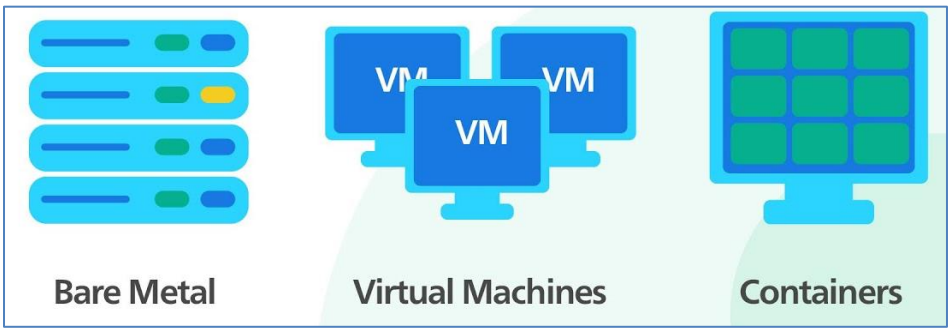


Figura 1: Formas de instalação de uma Stack de Desenvolvimento

A *Figura 1* apresenta três formas de instalação e execução de uma *Stack de Desenvolvimento*. À esquerda, o modelo *Bare Metal* representa o ambiente instalado diretamente no hardware, sem camadas intermediárias — o que oferece alto desempenho, porém pouca flexibilidade e isolamento. No centro, o modelo de *Máquinas Virtuais (Virtual Machines)* adiciona um hipervisor que permite executar múltiplos sistemas operacionais sobre o mesmo hardware, cada um com seu próprio conjunto de recursos, mas com maior consumo e menor velocidade de inicialização. À direita, os *Containers* representam a abordagem moderna e leve: eles compartilham o mesmo sistema operacional base, isolando apenas os processos e dependências de cada aplicação. Isso resulta em inicialização quase instantânea, uso eficiente de recursos e facilidade de portabilidade entre diferentes ambientes de desenvolvimento.

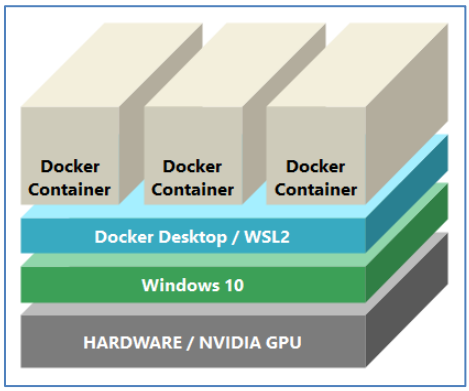


Figura 2: Representação visual da interação entre sistema operacional e Docker

A *Figura 2* ilustra de forma simplificada como o *Docker* interage com o sistema operacional e o hardware em um ambiente *Windows*. Na base está o *hardware físico* (como processador e GPU), sobre o qual o *Windows* é executado. Acima dele, o *Docker Desktop* trabalha em conjunto com o *WSL2 (Windows Subsystem for Linux 2)*, que fornece o kernel Linux necessário para o funcionamento dos containers. Essa camada permite que o Docker opere de maneira nativa dentro do Windows, sem depender de máquinas virtuais pesadas. Por fim, sobre o *Docker Desktop / WSL2*, são executados os *containers Docker*, que representam os ambientes isolados de aplicação, cada um com suas dependências, bibliotecas e serviços, sem interferir no sistema principal. Essa arquitetura garante isolamento, portabilidade e desempenho eficiente no desenvolvimento de aplicações.

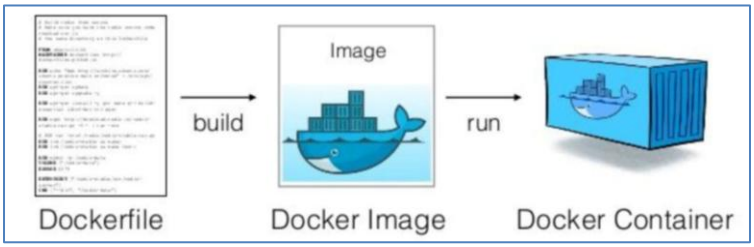


Figura 3: Processo de criação de um container Docker

A *Figura 3* representa o *processo de criação de um container Docker*. Tudo começa com o *Dockerfile*, um arquivo de texto que contém as instruções necessárias para montar o ambiente, como o sistema base, dependências, variáveis e comandos de inicialização. A partir desse arquivo, o comando *build* é executado, gerando uma *imagem Docker* — uma espécie de modelo ou "fotografia" do ambiente configurado. Essa imagem contém tudo o que é preciso para que a aplicação funcione de forma independente do sistema operacional. Quando a imagem é executada com o comando *run*, ela dá origem a um *container Docker*, que é uma instância em execução dessa imagem. Cada container é isolado, leve e descartável, podendo ser iniciado, parado e removido conforme a necessidade, garantindo portabilidade e reprodutibilidade no desenvolvimento de software.

- **Stack local:** cada desenvolvedor instala manualmente Node.js, bibliotecas e ferramentas no próprio computador.
    - Vantagem: desempenho máximo.
    - Desvantagem: risco de incompatibilidades entre ambientes.

- **Stack containerizada (ex.: Docker):** o ambiente é descrito em um Dockerfile, garantindo que todos usem a mesma configuração, independente do sistema operacional.
    - Vantagem: consistência total e facilidade de onboarding.
    - Desvantagem: maior uso de recursos e curva inicial de aprendizado.

1.4. Importância do stack

Ter uma stack de desenvolvimento bem definida:

- Reduz erros e conflitos entre máquinas diferentes;
- Acelera o onboarding de novos membros da equipe;
- Facilita a automação (CI/CD, testes e deploys);
- Melhora a padronização e manutenção do código;
- Torna o ambiente reproduzível, rastreável e escalável.
- **Vue 3 + Vite** — base do app e servidor de desenvolvimento rápido.
    - Vue 3: framework reativo que sincroniza dados e interface automaticamente.
    - Vite: dev server com hot-reload (HMR) e build rápido. Configurado para expor em 0.0.0.0:5173 e HMR em 24678.
- **Vue Router** — navegação entre telas sem recarregar a página.
    - Define rotas (por exemplo, / e /kanban).
    - Usa `<router-link>` e `<router-view>` para trocar componentes.
- **Pinia** — estado global e centralização de dados.
    - Armazena tarefas, filtros e lógica de atualização (actions, getters).
    - Facilita compartilhamento de dados entre componentes/páginas.
- **Tailwind CSS** — estilização utilitária rápida.
    - Classes utilitárias (por exemplo, p-4, bg-gray-100, rounded) para montar o layout sem CSS manual.
    - Responsivo e consistente para prototipagem e produção.
- **ESLint + Prettier** — qualidade de código e formatação.
    - ESLint: regras de padrão de código (inclui plugin Vue).
    - Prettier: formatação automática e consistente do projeto.

- **Vitest (unit)** — testes unitários rápidos.
  - Executa testes de componentes e funções com jsdom.
  - Integração pronta no Vite, com comando `npm run test`.
- **Playwright (E2E)** — testes ponta a ponta.
  - Simula o navegador (Chromium, Firefox, WebKit) para testar fluxos reais. Comandos `npm run test:e2e` e `npm run test:e2e:ui`.
- **Husky + lint-staged** — validação antes do commit.
  - Roda Prettier/ESLint nos arquivos alterados e testes unitários do Vitest; se falhar, o commit é bloqueado.
- **Axios** — biblioteca JavaScript para realizar requisições HTTP (como GET, POST, PUT, DELETE) de forma simples e moderna.
  - Em geral, é usada para comunicação com o backend — ou seja, é o que permite que o Vue envie e receba dados de um servidor.
  - Aplicações Vue.js, React e Node.js usam Axios para conectar o frontend com APIs.
  - Principais recursos:
    - Buscar e enviar dados de/para servidores REST (por exemplo, https://api.meusite.com/tarefas);
    - Suporta promises (async/await);
    - Faz tratamento automático de JSON;
    - Permite configurar interceptores, cabeçalhos (headers) e autenticação com token.

      Interceptores (ou *interceptors*) são funções especiais do Axios que "interceptam" uma requisição antes de ela sair do navegador ou antes de a resposta chegar do servidor. Servem para modificar, monitorar ou tratar erros automaticamente, sem precisar repetir o mesmo código em cada chamada axios.get() ou axios.post().
- **Lodash** — biblioteca de funções utilitárias que ajuda a manipular arrays, objetos, strings e números de forma mais simples, legível e eficiente.
  - Serve para trabalhar com dados de maneira mais prática e elegante, evitando reescrever funções comuns (como ordenação, filtragem e cópias de objetos).
  - Principais recursos:
    - Métodos prontos para ordenar (`_.sortBy`), filtrar (`_.filter`), agrupar (`_.groupBy`), clonar (`_.cloneDeep`), mesclar (`_.merge`) e comparar dados;
    - Reduz a quantidade de código repetitivo e erros em manipulação de dados complexos.

- **VS Code Dev Containers** — abre a pasta dentro do container.
    - Extensão do VS Code que monta o ambiente Docker com extensões (Volar, TypeScript Vue Plugin, Prettier, ESLint, Playwright).
    - Terminal integrado já dentro do container.

2. Requisitos tecnológicos

2.1. Comparação entre modelos com e sem Docker

A tabela seguinte compara o processo de criação e manutenção de um ambiente de desenvolvimento front-end com Vue.js, mostrando as diferenças entre configurá-lo diretamente no sistema (sem Docker) e dentro de containers (com Docker e VS Code Dev Containers).

Tópico	Sem Docker (host)	Com Docker (Dev Container / Compose)
Pré-requisitos	Node.js, npm/pnpm e Git instalados localmente	Docker Desktop/Engine (+ WSL2 no Windows) e Git
Criação do projeto	`npm create vue@latest + npm install`	`docker compose run --rm web npm create vue@latest .`
Execução	`npm run dev (porta 5173)`	`docker compose up` (mapeia 5173 e HMR 24678)
Paridade entre máquinas	Variável (depende da instalação de cada um)	Alta (mesma imagem e dependências)
Conflitos de versão	Comuns (Node 18 vs 20, libs nativas)	Eliminados (versão controlada na imagem)
Performance	Melhor em macOS/Linux	Boa no WSL2; um pouco mais lenta no Windows sem WSL2
Serviços adicionais	Precisam ser instalados localmente	Podem ser adicionados via `docker-compose`
Debug no VS Code	Nativo (direto no host)	Dev Container com terminal e extensões configuradas
Dependências globais	Instaladas no host (podem conflitar)	Isoladas no container
Onboarding	Demorado (várias instalações manuais)	Rápido (clonar + `docker compose up`)

2.2. Ambiente sem Docker

Configuração e execução local tradicional (diretamente no sistema operacional):

```
npm create vue@latest meu-app
cd meu-app
npm install
npm run dev
```

Instalação de bibliotecas adicionais:

```
npm install axios
```

O servidor de desenvolvimento será iniciado e poderá ser acessado em http://localhost:5173.

Vantagens:

- Melhor desempenho de I/O
- Menor consumo de recursos do sistema
- Simples para pequenos projetos

Desvantagens:

- Conflitos de versão entre máquinas
- Dependências instaladas diretamente no sistema
- Dificuldade para padronizar ambientes de turma ou equipe

2.3. Ambiente com Docker

Configuração e execução dentro de container (Docker + VS Code Dev Containers):

```
git clone <seu-repo.git>
cd seu-repo
docker compose up --build
```

Para trabalhar no VS Code Dev Container:

```
F1 → Dev Containers: Reopen in Container
npm run dev -- --host 0.0.0.0 --port 5173 --strictPort
```

Instalação de bibliotecas dentro do container:

```
docker compose run --rm web npm install axios
```

Vantagens:

- Ambiente idêntico para todos os desenvolvedores
- Nenhum Node.js precisa ser instalado localmente
- Evita conflitos de dependência e versão
- Paridade com CI/CD
- Possibilidade de adicionar serviços (Banco de Dados, API, Redis) via docker-compose

Desvantagens:

- Maior uso de CPU/memória
- Pode ter I/O mais lento no Windows (mitigável com WSL2)
- Requer conhecimento básico de Docker

2.4. Quando escolher cada abordagem

- Use Docker quando precisar de padronização entre máquinas, ambiente isolado, integração com pipelines de CI/CD ou vários serviços integrados.
- Não use Docker quando trabalhar sozinho, em projeto simples e quiser máximo desempenho local.

2.5. Dicas práticas para ambos os ambientes

- No Windows, habilite e use o WSL2 para melhor desempenho com Docker.
- Monte o volume node_modules no docker-compose.yml para acelerar reinstalações.
- Sempre exponha --host 0.0.0.0 e mapeie as portas 5173 (servidor) e 24678 (HMR).
- Use o VS Code com a extensão Dev Containers para manter consistência de ambiente.

3. Guia para Stack "Dockerizada"

Um ambiente dockerizado de desenvolvimento é aquele em que todo o processo de programação e execução de um projeto — incluindo dependências, bibliotecas, serviços auxiliares e configurações — ocorre dentro de containers Docker. Containers são ambientes isolados que garantem que o projeto funcione da mesma forma em qualquer computador, independentemente do sistema operacional, eliminando erros causados por diferenças de versões ou configurações locais.

No Windows, o Docker depende do WSL2 (Windows Subsystem for Linux 2), que é um subsistema que permite ao Windows executar um kernel Linux real dentro do sistema. Isso é importante porque o Docker foi originalmente projetado para o Linux, e o WSL2 fornece a compatibilidade necessária para que os containers funcionem de forma nativa e eficiente. O WSL2 executa distribuições Linux leves (como Ubuntu ou Debian), com excelente integração de arquivos e desempenho próximo ao nativo.

O Docker Desktop, por sua vez, é a interface gráfica e o gerenciador central que permite criar, executar e monitorar containers de forma prática no Windows. Ele se conecta automaticamente ao WSL2, fornecendo ao desenvolvedor uma experiência unificada — é possível visualizar containers, imagens e volumes, configurar redes e até interagir com o Docker Hub (repositório online de imagens).

Durante o desenvolvimento, o uso de volumes Docker é fundamental: eles criam uma ponte entre os arquivos do projeto no Windows e o ambiente Linux dentro do container. Isso faz com que qualquer alteração feita no código local seja refletida instantaneamente dentro do container, sem precisar reconstruí-lo. Assim, o desenvolvedor mantém a velocidade de iteração de um ambiente local tradicional, mas com toda a consistência, isolamento e portabilidade de um ambiente dockerizado.

3.1. O que é o WSL2 e como configurá-lo no Windows 10/11

O WSL2 (Windows Subsystem for Linux 2) é uma tecnologia da Microsoft que permite executar um ambiente Linux completo dentro do Windows. Ele é essencial para o funcionamento do Docker Desktop em sistemas Windows modernos, pois fornece o kernel Linux necessário para containers.

Passos para instalar e configurar o WSL2:

a) Abra o PowerShell como administrador.
b) Execute o comando abaixo para instalar o WSL e o Ubuntu automaticamente:

```
wsl --install
```

c) Reinicie o computador quando solicitado. Após o reinício, o Ubuntu será configurado automaticamente.
d) Verifique se o WSL2 está ativo e configurado como versão padrão:

```
wsl --set-default-version 2
```

e) Liste as distribuições instaladas e confirme a versão:

```
wsl -l -v
```

f) Caso alguma distribuição ainda use WSL1, converta-a para WSL2:

```
wsl --set-version Ubuntu-20.04 2
```

g) Abra o aplicativo Ubuntu no menu Iniciar, crie um usuário e senha. O ambiente Linux estará pronto para uso com o Docker.

3.2. Instalação do Docker Desktop no Windows

O Docker Desktop é a forma mais prática de instalar e gerenciar containers no Windows. Ele integra o Docker Engine com o WSL2 e fornece uma interface gráfica intuitiva.

a) Acesse https://www.docker.com/products/docker-desktop.
b) Baixe o instalador (Docker Desktop Installer.exe).
c) Execute o instalador e marque a opção 'Use WSL 2 instead of Hyper-V'.
d) Após a instalação, reinicie o computador.
e) Abra o Docker Desktop e aguarde até que o daemon Docker seja iniciado.
f) Verifique a instalação com os comandos:

```
docker --version
docker compose version
docker run hello-world
```

Se aparecer a mensagem 'Hello from Docker!', a instalação foi concluída com sucesso.

3.3. Principais Comandos Docker

Lista de comandos mais utilizados:

- `docker pull <imagem>` — Baixa uma imagem do Docker Hub.
- `docker images` — Lista as imagens locais.
- `docker run <imagem>` — Cria e inicia um container.
- `docker ps` — Mostra containers em execução.
- `docker ps -a` — Lista todos os containers, inclusive parados.
- `docker stop <id>` — Para um container em execução.
- `docker rm <id>` — Remove um container.
- `docker rmi <imagem>` — Remove uma imagem.
- `docker build -t nome:versao .` — Cria uma imagem a partir do Dockerfile.
- `docker compose up --build` — Inicia serviços definidos no docker-compose.yml.
- `docker compose down` — Remove containers criados pelo Compose.
- `docker system prune` **-a** — Limpa imagens e containers não utilizados.

3.4. Docker Hub e Interação pelo Docker Desktop

O Docker Hub é o repositório oficial de imagens Docker, usado para armazenar e compartilhar imagens públicas ou privadas. Você pode acessá-lo tanto pela interface do Docker Desktop quanto pelo terminal.

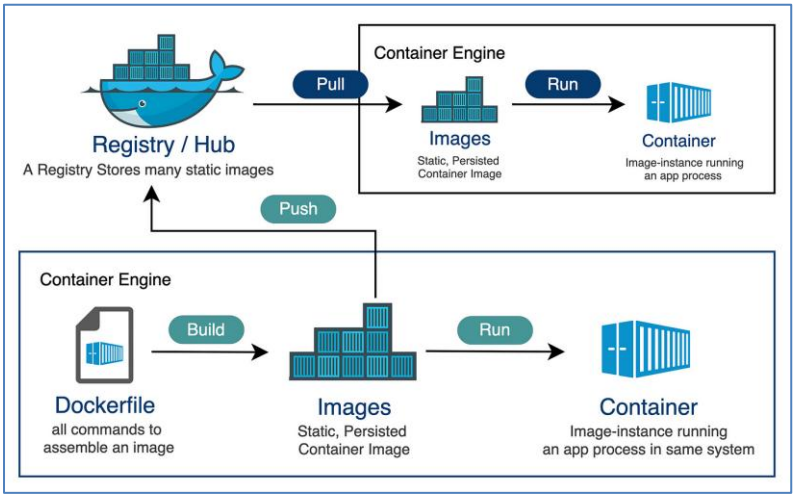


Figura 4: Representação visual da integração entre Docker Desktop e Docker Hub

A *Figura 4* ilustra o fluxo de integração entre o *Docker Desktop* e o *Docker Hub*, mostrando como as imagens são criadas, armazenadas e reutilizadas. O processo começa com o *Dockerfile*, que contém as instruções para construir uma imagem. O *Container Engine* executa o comando *build*, gerando uma *imagem Docker* — um pacote estático que contém o ambiente completo necessário para rodar uma aplicação. Essa imagem pode ser executada localmente com o comando *run*, criando um *container* em funcionamento. Quando a imagem está pronta para ser compartilhada, o desenvolvedor pode enviá-la ao *Docker Hub* por meio do comando *push*. No *Docker Hub* (ou outro *Registry*), a imagem fica

armazenada e disponível para ser baixada em qualquer outro ambiente através do comando *pull*. Esse ciclo de *build* → *push* → *pull* → *run* permite que equipes compartilhem ambientes padronizados, garantindo consistência entre desenvolvimento, testes e produção.

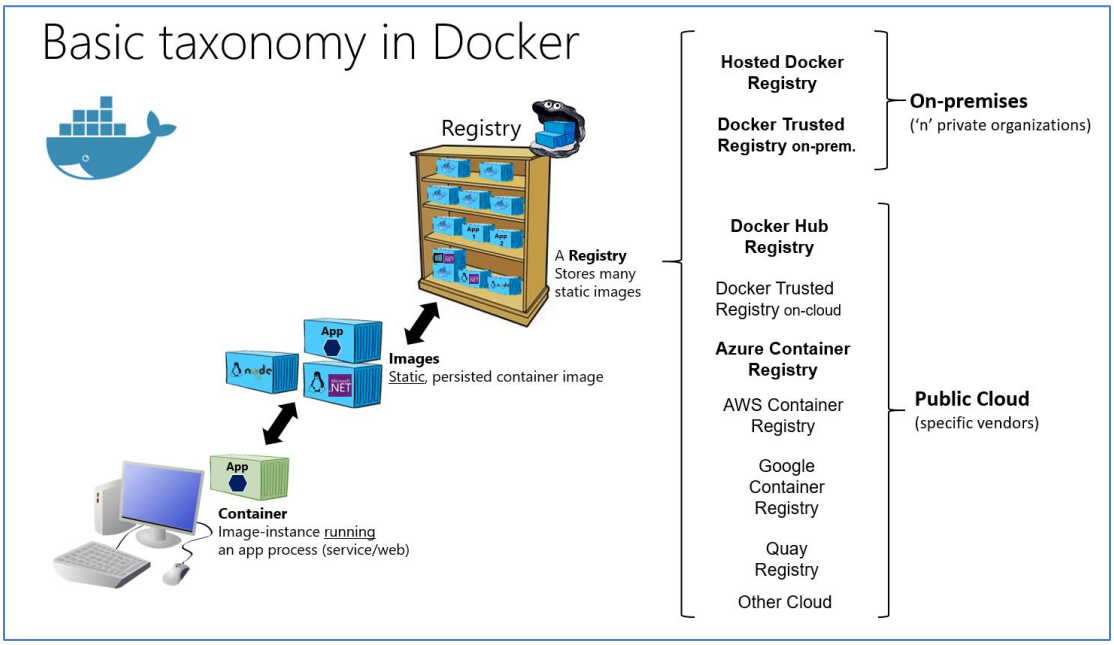


Figura 5: Taxonomia dos componentes Docker

A *Figura 5* apresenta a *taxonomia dos componentes Docker*, mostrando como os elementos principais se relacionam no ecossistema. Na base, temos o *container*, que é uma instância em execução de uma imagem e representa o ambiente onde a aplicação realmente roda. Acima dele estão as *imagens Docker*, que são pacotes estáticos contendo tudo o que é necessário para a execução do software — sistema base, bibliotecas, dependências e configurações. Essas imagens são armazenadas em um *Registry*, que funciona como um repositório de versões, similar a um "repositório Git" para containers. O *Docker Hub* é o exemplo mais conhecido de registry público, mas há também registries privados (*on-premises*) e hospedados em nuvem (*cloud*), como o *Azure Container Registry*, *AWS Container Registry* e *Google Container Registry*. Essa estrutura permite que imagens sejam criadas, compartilhadas e baixadas de maneira padronizada, facilitando o versionamento, a colaboração entre desenvolvedores e a integração contínua entre diferentes ambientes de desenvolvimento e produção.

Principais operações:

a) Login no Docker Hub:

```
docker login
```

b) Buscar imagens:

```
docker search node
docker pull node:20-alpine
```

c) Executar imagens localmente:

```
docker run -it node:20-alpine /bin/sh
```

d) Publicar suas imagens:

```
docker tag meuapp:1.0 usuario/meuapp:1.0
docker push usuario/meuapp:1.0
```

e) No Docker Desktop, é possível visualizar, gerenciar e excluir imagens diretamente pela aba Images. As imagens públicas podem ser usadas por qualquer pessoa, enquanto as privadas requerem autenticação.

f) Sobre as tags: cada imagem pode ter várias versões. A tag 'latest' é a padrão e representa a versão mais recente.